The process begins by collecting documents from a specified directory and converting them into manageable chunks that preserve contextual information. Since we've already explained how to do this step in the summarization web application for a single document, we can now extend this to perform over a list of documents.

Next, those chunks are transformed using an embedding model, which converts text into dense vector representations. Once vectorized, the document chunks are stored in a vector database like HNSWLib, which enables efficient semantic search. Each vector is accompanied by metadata such as the source document, page number, and processing timestamp, providing additional context for later retrieval.

We saw a similar example in the previous chapter using a `MemoryVectorStore` class. However, now we need to be able to store the indexes in a more permanent solution so that our web application can perform semantic retrieval. The HNSWLib vector store, which is part of the LangChain.js community, is a perfect solution for our needs. HNSWLib is a portable and easy-to-use store that saves the indexes in the filesystem, which can be loaded later on for similarity searching.

The following listing shows the key section of the indexing algorithm that processes each document from a directory and indexes them using the HNSWLib vector store.

Listing 7.8 Indexing documents using a vector store

```
import { HNSWLib } from '@langchain/community/vectorstores/hnswlib';
async indexDocumentsFromDirectory(documentDirectory) {
    const documents = [];

    // Find all PDF files in the directory
    const pdfFiles = fs
      .readdirSync(documentDirectory)
      .filter((file) =>
path.extname(file).toLowerCase() === '.pdf')
      .map((file) =>
path.join(documentDirectory, file));

    // Process each PDF document
    for (const filePath of pdfFiles) {
      const processedDocs = await this.processDocument(filePath);
      documents.push(...processedDocs);
    }

    // Create vector store from processed documents
    const vectorStore =
await HNSWLib.fromDocuments(documents, this.embeddings);

    console.log(`Indexed ${documents.length}
document chunks from ${pdfFiles.length} documents`);

    return vectorStore;
  }
```

Finds all PDF files in the directory

Processes each PDF document

Creates vector store from processed documents